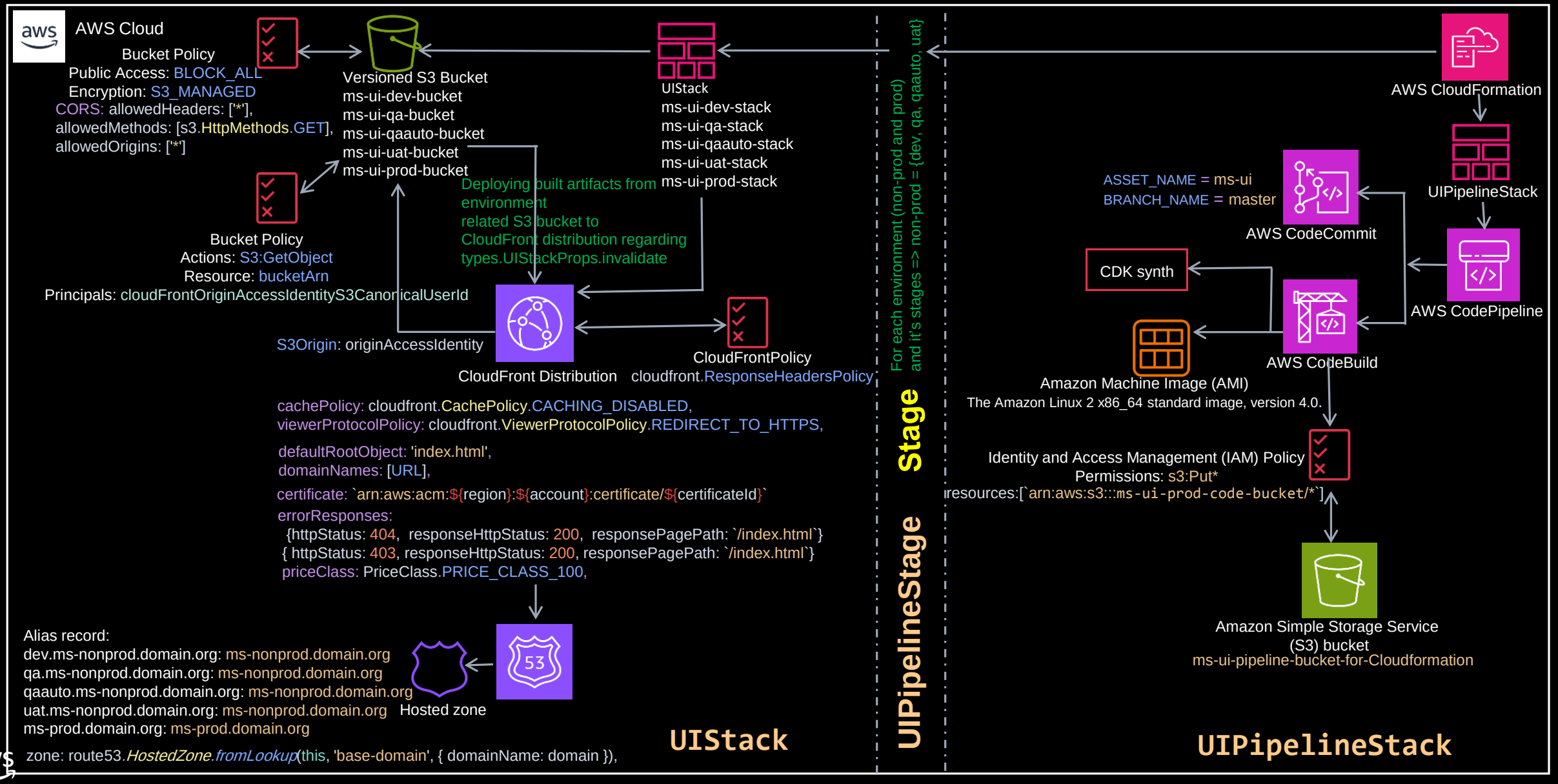
Max Montaseri

AWS Infrastructure as Code

UI (Front end)

aws

Stage: An abstract application modeling unit consisting of Stacks that should be deployed together. Derive a subclass of Stage and use it to model a single instance of your application. You can then instantiate your subclass multiple times to model **multiple copies of your application** which should **be deployed to different environments**.



types.ts

```
import { StackProps, StageProps } from 'aws-cdk-lib'

UIPipelineStackProps
  -vpcId: string
  -waves: Array<UIStageProps>
                -approval?: boolean
                -name: string
                -environments: Array<UIStackProps>
                                        -stackEnv: string
                                        -invalidate?: boolean
```

aws

config.ts

```
const MS_NAME='order' // MicroserviceName (Exp. order)
const ENV_PROD = { account: '????', region: 'us-east-1' }
// region, account => only for creating ACM certificate for cloudfront.Distribution
const PROD_VPCID = 'vpc-????'//'vpc-prod' // already should be created in account
const PROD_ACM_CERT_ID = ????????'
const PROD_ROOT_DOMAIN = `${MS_NAME}-prod.domain.org`
const ENV_NONPROD = { account: '????', region: 'us-east-1' }
const NONPROD_VPCID = 'vpc-????'
const NONPROD_ACM_CERT_ID = '????'
const NONPROD_ROOT_DOMAIN = `${MS_NAME}-nonprod.domain.org`
export const PIPELINE_NAME = `${MS_NAME.toUpperCase}-UIPipeline`
export const BRANCH_NAME = 'master'
export const ASSET_NAME = `${MS_NAME}-ui`// Used in custom domain name => The custom domain name for your API.
Uppercase letters are not supported
export const DEPENDENCY_TRACK_S3 = `${MS_NAME}-ui-pipeline-bucket-for-Cloudformation`
// this UI Pipekine will save the UIPipelineStack built artifacts (by CodePipekine) for executing by AWS
Cloudformation
export const getTags = (env: string) => ({
  asset_id: ASSET_NAME,
  system_id: `${MS_NAME}`,
  accessibility: 'private',
  data_classification: 'proprietary',
  env_intent: env,
  engineer: 'frontend@company.org',
})
export const getEnvironmentConfig = (env: string) =>
  env === 'prod'
    ? {
        certificateId: PROD_ACM_CERT_ID,
        domain: PROD_ROOT_DOMAIN,
        vpcId: PROD_VPCID,
      }
    : {
        certificateId: NONPROD_ACM_CERT_ID,
        domain: NONPROD_ROOT_DOMAIN,
        vpcId: NONPROD_VPCID,
```

aws

config.ts

```
export const PIPELINE_CONFIG = {
  env: ENV_NONPROD,
  tags: getTags('nonprod'),
  vpcId: NONPROD_VPCID,
  waves: [
    {
      environments: [{ stackEnv: 'dev' }],
      logLevel: 'debug',
      name: 'dev-stage',
      env: ENV_NONPROD,
      tags: getTags('dev'),
    },
    {
      environments: [{ stackEnv: 'qa' }],
      logLevel: 'debug',
      approval: true,
      name: 'qa-stage',
      env: ENV_NONPROD,
      tags: getTags('qa'),
    },
    {
      environments: [{ stackEnv: 'qaauto' }, { stackEnv: 'uat' }],
      approval: true,
      name: 'preprod-stage',
      env: ENV_NONPROD,
      tags: getTags('preprod'),
    },
    {
      environments: [{ stackEnv: 'prod', pitr: true }],
      approval: true,
      name: 'prod-stage',
      env: ENV_PROD,
      tags: getTags('prod'),
    },
  ],
}
```

aws

UIPipelineStack (CodeBuildStep, CodePipelineSource.codeCommit and CodePipeline)

will be executed whenever `PIPELINE_CONFIG` or `config.ts` configurations are changed in CodeCommit repository.
Then reads the pipeline's stages (waves) and their environments dev|qa|qaauto|uat|prod (StackEnv, Domain) from `PIPELINE_CONFIG.waves`, Finally invokes the UIStack for provisioning UI provider services for each environment

```
const app = new cdk.App(); new UIPipelineStack(app, PIPELINE_NAME, PIPELINE_CONFIG);
```

UIPipelineStack

```
const { waves, vpcId } = props || {}
    const vpc = Vpc.fromLookup(this, 'vpc', { vpcId })

    const pipeline = new CodePipeline(this, 'Pipeline', {
      selfMutation: true,
      pipelineName: PIPELINE_NAME,
      crossAccountKeys: true,
      codeBuildDefaults: {
      /*  CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-msName
      and doesn't exist in InfrastructurePipelineStack    */
        buildEnvironment: {
          // see https://docs.aws.amazon.com/codebuild/latest/userguide/build-env-ref-available.html
          buildImage: LinuxBuildImage.AMAZON_LINUX_2_4,
        },
      },
```

UIPipelineStack

```
synth: new CodeBuildStep('Synth', {
      // Synth => will take our codes then generate AWS CloudFormation templates
      // => we can see the generated templates=> this happens always we deploy!
        projectName: ASSET_NAME,
        input: CodePipelineSource.codeCommit(Repository.fromRepositoryName(this, 'repo', ASSET_NAME), BRANCH_NAME),
        commands: [
          // "node -v",
          "npm ci",
          "npm test",
          "npm run build",
          "npx cdk synth",
        ],
        rolePolicyStatements: [
        // CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-msName
        // and doesn't exist in InfrastructurePipelineStack
          new PolicyStatement({
            effect: Effect.ALLOW,
            actions: ['s3:Put*'],
            resources: [`arn:aws:s3:::${DEPENDENCY_TRACK_S3}/*`],
          }),
        ],
```

UIPipelineStack

```
        partialBuildSpec: BuildSpec.fromObject({
        // CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-msName
        // and doesn't exist in InfrastructurePipelineStack
          reports: {
            coverageReports: {
              files: 'clover.xml',
              'file-format': 'CLOVERXML',
              'base-directory': 'coverage',
            },
            jestReports: {
              files: 'unitTestReport.xml',
              'file-format': 'JUNITXML',
              'base-directory': 'coverage',
            },
          },
        }),
        vpc,
        // vpc: Vpc.fromLookup(this, "vpc", { vpcId }),
      }),
    })

    waves.forEach((w) => {
      const { environments, approval, name = '', ...common } = w
      const wave = pipeline.addWave(name, {
        pre: approval ? [new ManualApprovalStep(`promoteto${name}`)] : undefined,
      })

      environments.forEach((e) => {
        wave.addStage(new UIPipelineStage(this, `${ASSET_NAME}-${e.stackEnv}`, { ...common, ...e }))
      })
    })
```

aws

UIStack.ts

UIStack will invoke *config.getEnvironmentConfig(stackEnv)* for each environment/stackEnv
so certificateId, domain and vpcId would be set for that environment/stackEnv

in **UIStack.ts**:

1- An S3 bucket will be created for each environment => *bucketName: `${ASSET_NAME}-${stackEnv}`,*

2- for CloudFront's access/fetch requests, a S3 bucket access policy (principal: *cloudFrontOriginAccessIdentityS3CanonicalUserId*) will be created for each environment's S3 bucket

3- A CloudFront ResponseHeadersPolicy will be created for each environment/stackEnv
=> *responseHeadersPolicyName: `securityCacheControlPolicy-${stackEnv}`,*

4- Only for Price Class 100 (least expensive regions): US, Mexico, Canada, Europe and Israel
A CloudFront distribution with *CachePolicy.CACHING_DISABLED* will be created with each environment's S3 bucket as origin
=> *origin: new origins.S3Origin(appBucket, { originAccessIdentity }),*

5- A Route53 Cname record in a private zone will be created for this CloudFront distribution

Zone name: `${stackEnv}.${domain}`

domain:
PROD_ROOT_DOMAIN = `${MS_NAME}-prod.domain.org`
NONPROD_ROOT_DOMAIN = `${MS_NAME}-nonprod.domain.org`

6- Deploying built artifacts from environment related S3 bucket to CloudFront distribution regarding *types.UIStackProps.invalidate*